

Aula 09

Recursão



Algoritmos e Estrutura de Dados II

2º Semestre – CDN



Prof. Dr. Dilermando Piva Jr.

Conteúdo Programático - Planejamento

Semana	Data	Temas/Atividades
1	07/08	Acolhimento e Boas-vindas! Introdução a Disciplina. Formas de Avaliação e Percorso Pedagógico.
2	14/08	Tipo de dado abstrato. Introdução a Estrutura de Dados.
3	21/08	Complexidade de Algoritmos
4	28/08	Vetores não-Ordenados e busca sequencial
5	04/09	Vetores Ordenados e busca binária
6	11/09	Revisão de Programação Orientada a Objetos (POO)
7	18/09	Pilhas
8	25/09	Filas
9	02/10	Listas encadeadas
10	09/10	Recursão
11	16/10	Primeira Avaliação Formal (P1). Correção da Avaliação após o intervalo.
12	18/10	Algoritmos de Ordenação
13	23/10	Algoritmos de Ordenação
14	30/10	Árvores
15	06/11	Grafos
16	13/11	Segunda Avaliação Formal (P2). Correção da Avaliação após o intervalo
17	27/11	Apresentação PI do curso de CDN
18	04/12	Tabela Hash (tabela de espalhamento) – Tópico extra.
19	11/12	Exame / Avaliação Substitutiva. Correção da Avaliação após o intervalo. Finalização Disciplina
20	18/12	Finalização da disciplina.

Recursividade...

Uma função que chama a si mesma...

Você conhece as Bonecas Matryoshka?



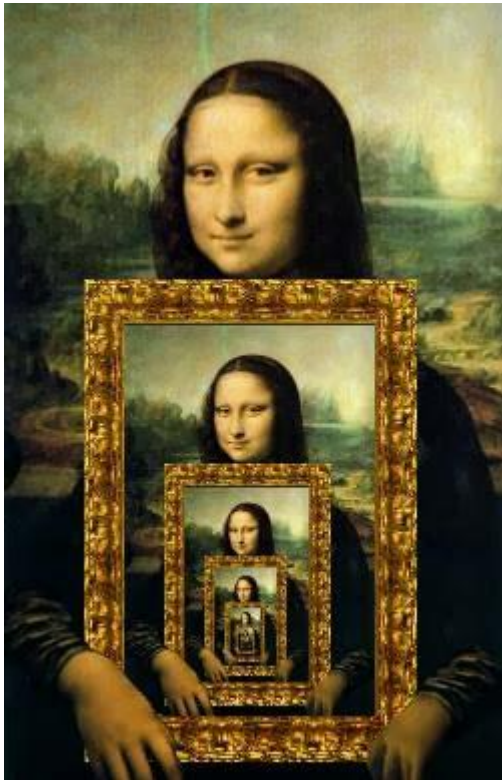
Matrioskas

Você já sonhou que estava sonhando?



Recursividade...

- Propriedade de uma função chamar a si mesma.
- Necessariamente, toda função recursiva deve ter uma ou mais condições de parada.



Exemplo clássico:
- Cálculo FATORIAL

Fatorial de 5

$$5 * 4 * 3 * 2 * 1 = 120$$

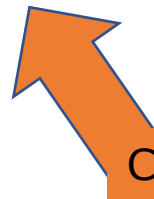
Fatorial de 0 = 1

Fatorial de 1 = 1

Recursividade

- Cálculo Fatorial

```
def fatorial(n):  
    if n > 1:  
        return n * fatorial(n-1)  
    else:  
        return 1
```



CONDIÇÃO DE PARADA !!

Recursividade

- Cálculo Fatorial

$N = 5$

$N *$

```
def fatorial(n):  
    if n > 1:  
        return n * fatorial(n-1)  
    else:  
        return 1
```



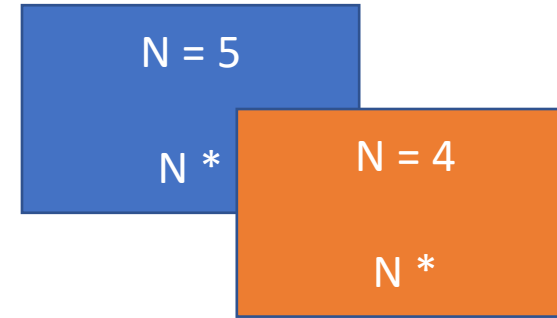
CONDIÇÃO DE PARADA !!

Recursividade

- Cálculo Fatorial

```
def fatorial(n):  
    if n > 1:  
        return n * fatorial(n-1)  
    else:  
        return 1
```

CONDIÇÃO DE PARADA !!

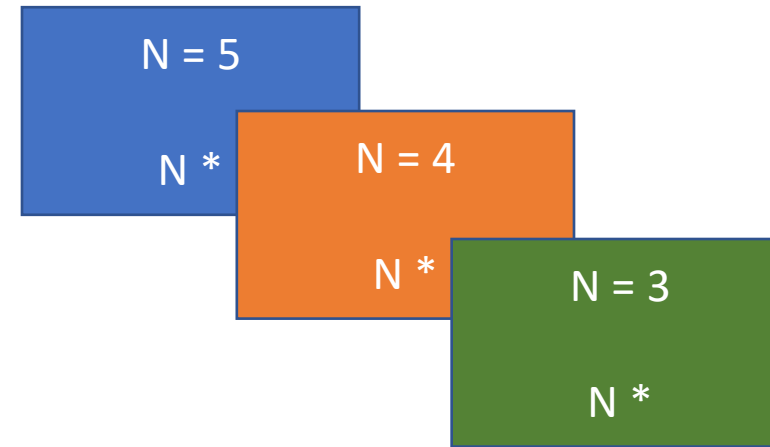


Recursividade

- Cálculo Fatorial

```
def fatorial(n):  
    if n > 1:  
        return n * fatorial(n-1)  
    else:  
        return 1
```

CONDIÇÃO DE PARADA !!

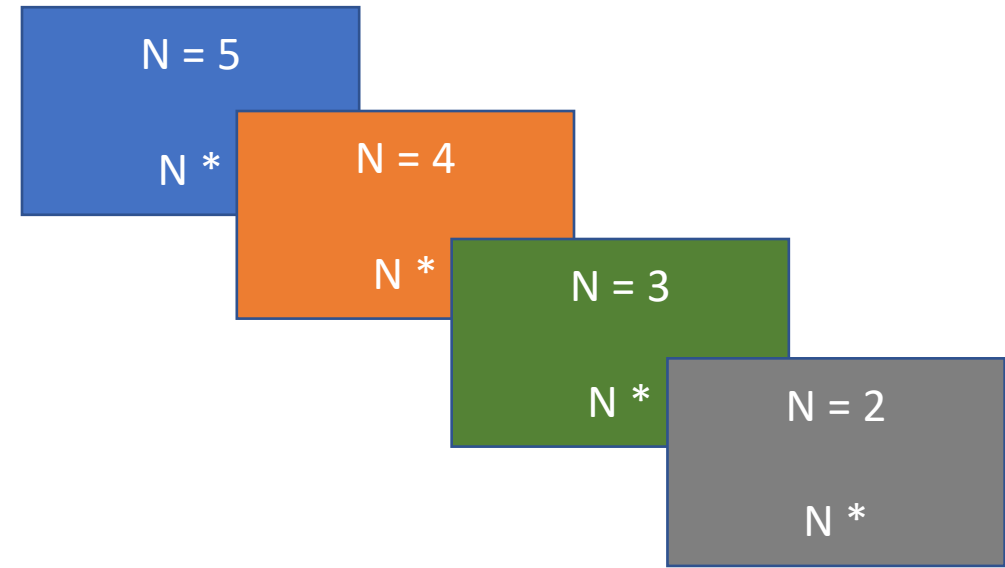


Recursividade

- Cálculo Fatorial

```
def fatorial(n):  
    if n > 1:  
        return n * fatorial(n-1)  
    else:  
        return 1
```

CONDIÇÃO DE PARADA !!

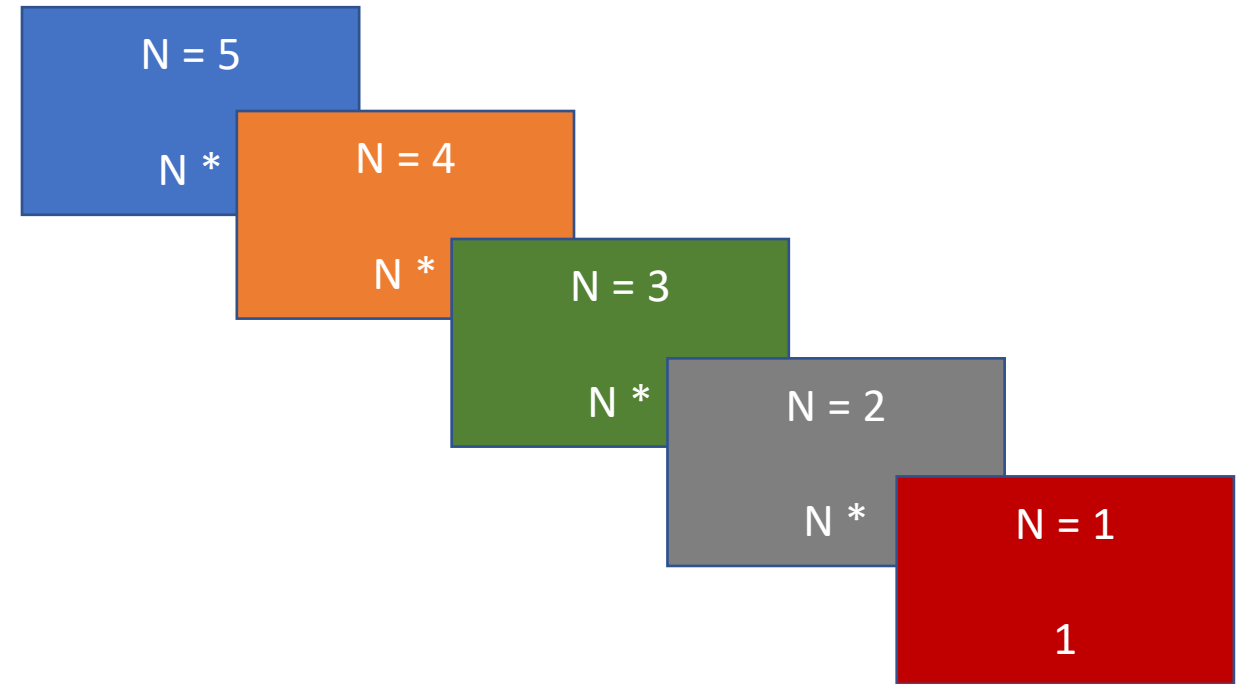


Recursividade

- Cálculo Fatorial

```
def fatorial(n):  
    if n > 1:  
        return n * fatorial(n-1)  
    else:  
        return 1
```

CONDIÇÃO DE PARADA !!

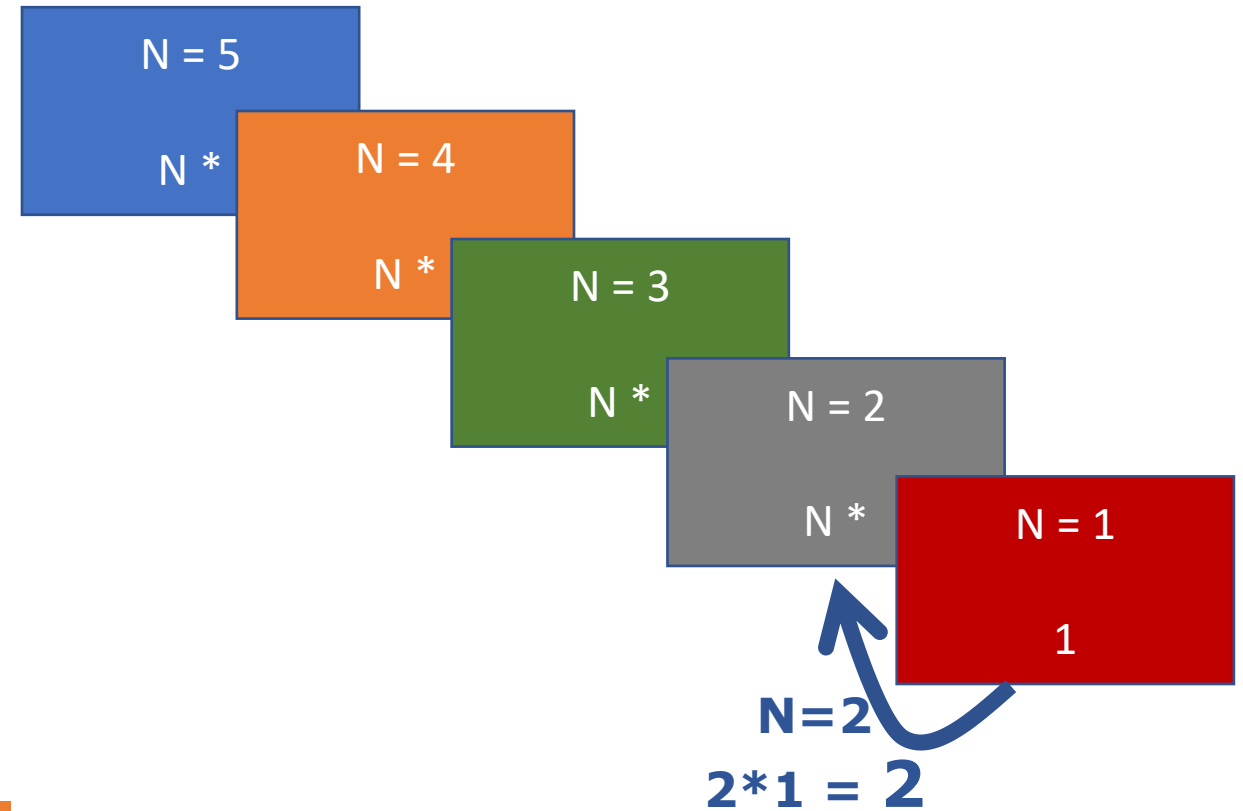


Recursividade

- Cálculo Fatorial

```
def fatorial(n):  
    if n > 1:  
        return n * fatorial(n-1)  
    else:  
        return 1
```

CONDIÇÃO DE PARADA !!

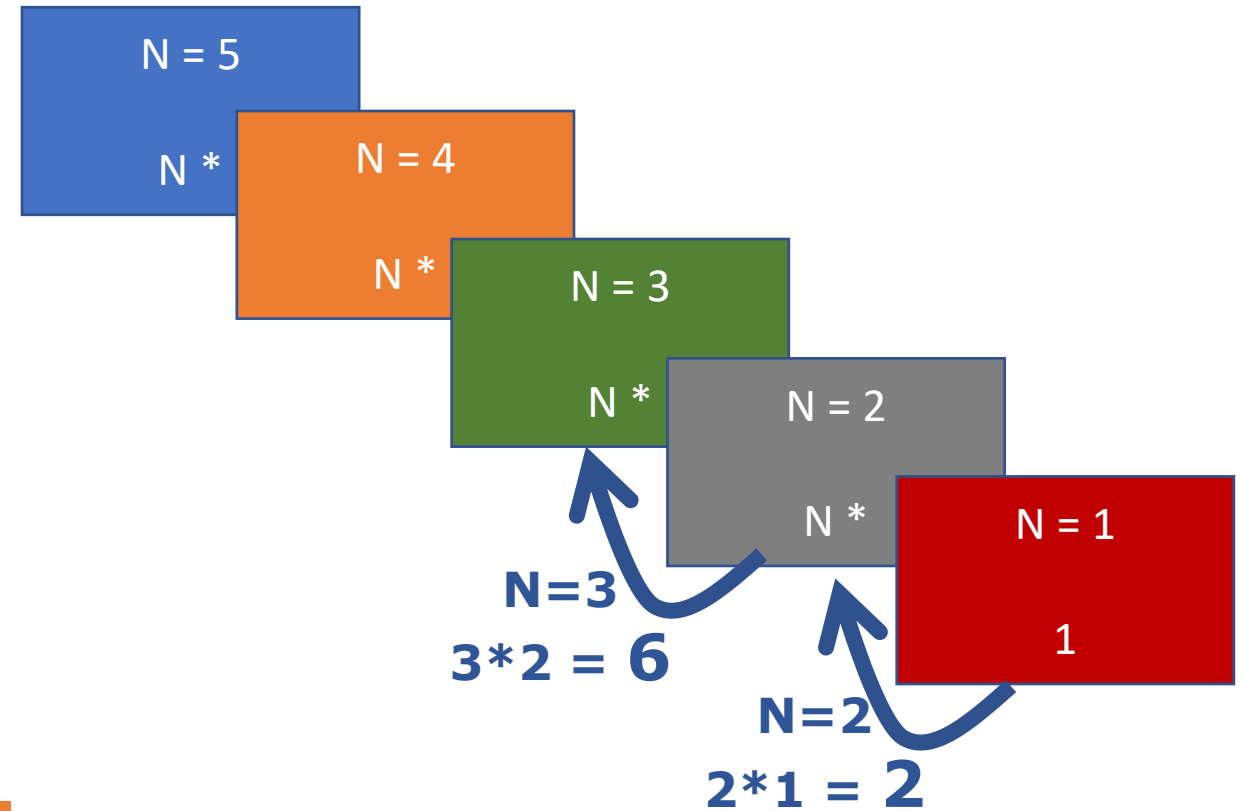


Recursividade

- Cálculo Fatorial

```
def fatorial(n):  
    if n > 1:  
        return n * fatorial(n-1)  
    else:  
        return 1
```

CONDIÇÃO DE PARADA !!

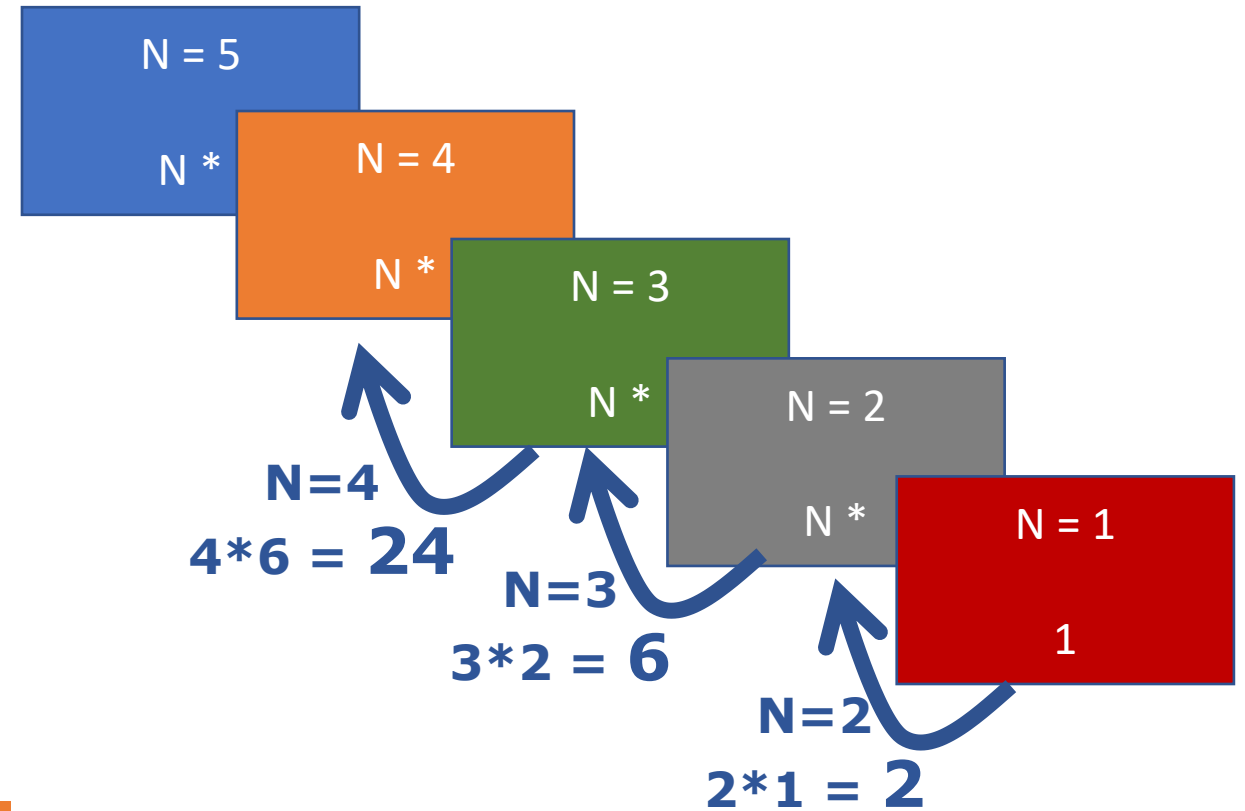


Recursividade

- Cálculo Fatorial

```
def fatorial(n):  
    if n > 1:  
        return n * fatorial(n-1)  
    else:  
        return 1
```

CONDIÇÃO DE PARADA !!

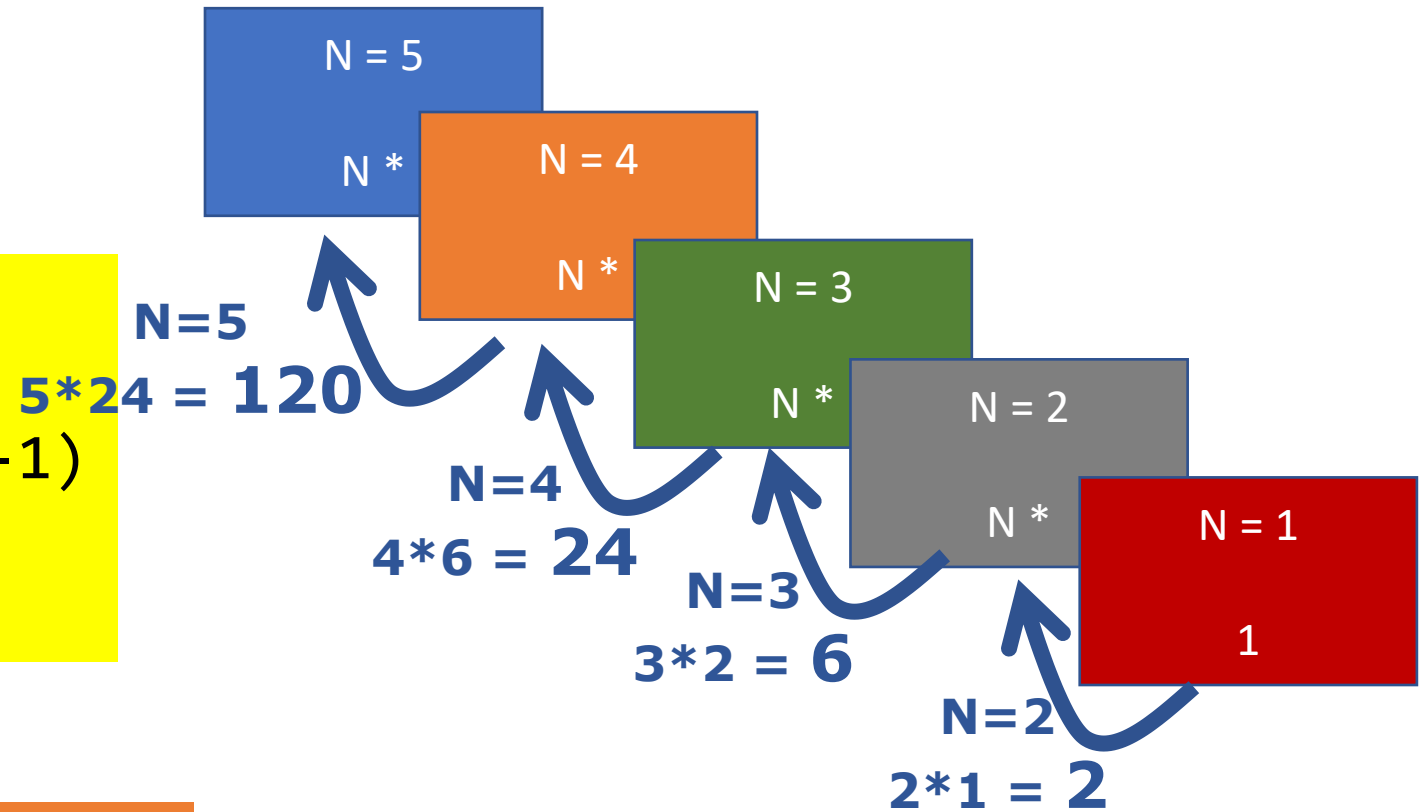


Recursividade

- Cálculo Fatorial

```
def fatorial(n):  
    if n > 1:  
        return n * fatorial(n-1)  
    else:  
        return 1
```

CONDIÇÃO DE PARADA !!



Recursividade

- Cálculo Fatorial

```
def fatorial(n):  
    if n > 1:  
        return n * fatorial(n-1)  
    else:  
        return 1
```

```
def fatorial(n):  
    return (n * fatorial(n-1)) if n > 1 else 1
```

Um algoritmo exponencial: Fibonacci recursivo (1 de 3)

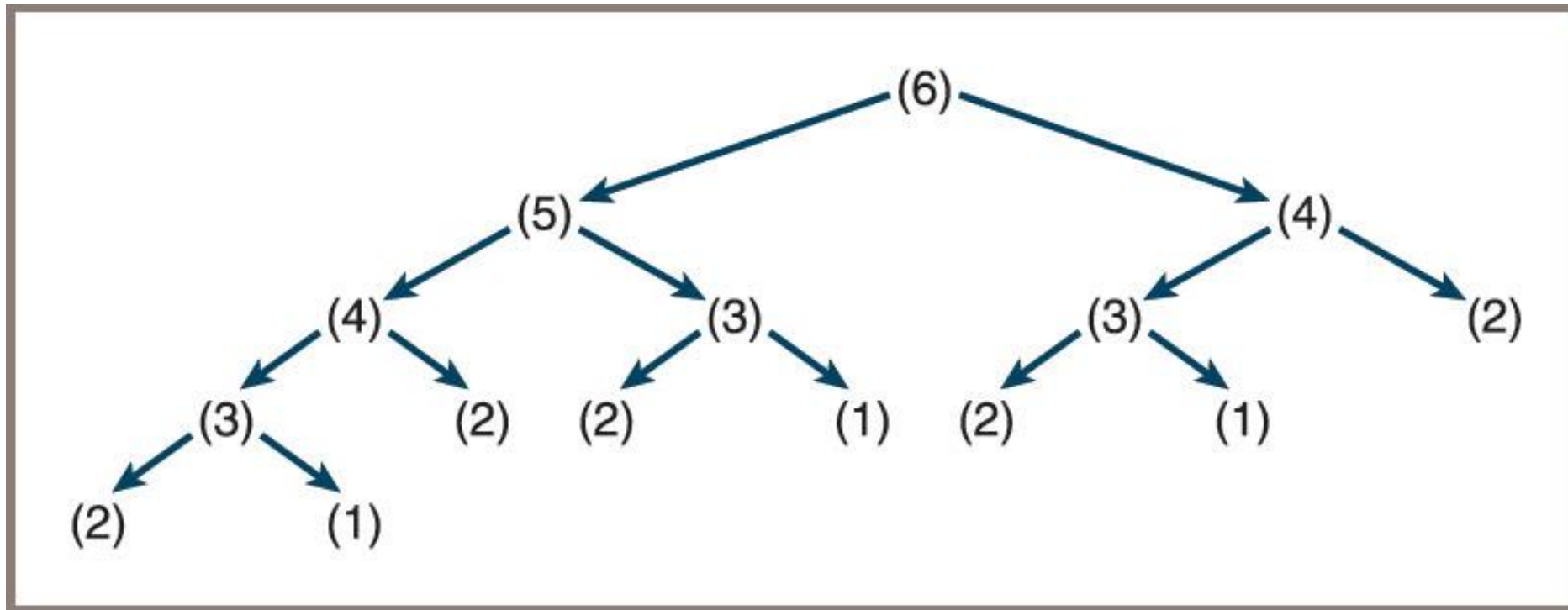
Código para a função Fibonacci:

```
def fib(n):  
    """A função Fibonacci recursiva."""  
    if n < 3:  
        return 1  
    else:  
        return fib(n - 1) + fib(n - 2)
```

Um algoritmo exponencial: Fibonacci recursivo (2 de 3)

A Figura abaixo mostra as chamadas envolvidas ao se usar a função recursiva para calcular o sexto número de Fibonacci.

Uma árvore de chamadas para fib(6)



Um algoritmo exponencial: Fibonacci recursivo (3 de 3)

- Algoritmos exponenciais geralmente são impraticáveis de serem executados com qualquer problema, exceto de tamanhos muito pequenos.
- Embora o design do Fibonacci recursivo seja elegante,
 - Há uma versão menos bela, mas muito mais rápida, que usa um laço para executar em tempo linear. (EM SEGUIDA)
- Como alternativa, funções recursivas que são chamadas repetidamente com os mesmos argumentos, como a função Fibonacci, podem se tornar mais eficientes por meio de uma técnica chamada **memorização**:
 - O programa mantém uma tabela dos valores para cada argumento usado com a função.
 - Antes de a função calcular recursivamente um valor para determinado argumento, ela verifica na tabela se o argumento já tem um valor
 - Nesse caso, o valor é simplesmente retornado.
 - Do contrário, o cálculo continua e o argumento e o valor são adicionados à tabela posteriormente.

Um algoritmo exponencial: Fibonacci recursivo (3 de 3)

- Algoritmos exponenciais geralmente são impraticáveis de serem executados com qualquer problema, exceto de tamanhos muito pequenos.
- Embora o design do Fibonacci recursivo seja elegante,
 - Há uma versão menos bela, mas muito mais rápida, que usa um laço para executar em tempo linear. (EM SEGUIDA)
- Como alternativa, funções recursivas que são chamadas repetidamente com os mesmos argumentos, como a função Fibonacci, podem se tornar mais eficientes por meio de uma técnica chamada memoização.
 - O programa mantém uma tabela com os resultados de cada chamada da função.
 - Antes de a função calcular um novo resultado, ela verifica na tabela se o argumento já foi calculado.
 - Nesse caso, o valor é simplesmente retornado.
 - Do contrário, o cálculo continua normalmente e o resultado é armazenado na tabela para uso futuro.

Complexidade:
 $O(n!)$

Convertendo Fibonacci em um algoritmo linear (1 de 2)

- o novo algoritmo inicia um laço se n é pelo menos o terceiro número de Fibonacci:
 - Esse número será pelo menos a soma dos dois primeiros ($1 + 1 = 2$).
- O laço calcula essa soma e, em seguida, realiza duas substituições:
 - O primeiro número se torna o segundo e o segundo se torna a soma recém-calculada.
- O laço conta de 3 a n .
- A soma no final do laço é o n -ésimo número de Fibonacci.

Convertendo Fibonacci em um algoritmo linear (2 de 2)

Pseudocódigo para o algoritmo:

```
Configura sum como 1
Configura first como 1
Configura second como 1
Configura count como 3
Enquanto count <= N
    Configura sum como first + second
    Configura first como second
    Configura second como sum
    Incrementa count
```

Exemplo de uso:

N = 10 # substitua pelo valor desejado

resultado = fibonacci(N)

print(f"O {N}-ésimo número da sequência de Fibonacci é: {resultado}")

```
def fibonacci(N):
    if N <= 0:
        return "O valor de N deve ser maior que 0"
    elif N == 1:
        return 1
    elif N == 2:
        return 1

    sum, first, second = 1, 1, 1
    count = 3

    while count <= N:
        sum = first + second
        first = second
        second = sum
        count += 1

    return sum
```



Convertendo Fibonacci em um algoritmo linear (2 de 2)

Usando o algoritmo, agora linear, para mostrar os n números de fibonacci

```
# Exemplo de uso:
```

```
# n = a qtd de números da sequencia
```

```
n = int(input("digite o valor de n: "))
```

```
for i in range(1, n+1):  
    print(fibonacci(i))
```

```
def fibonacci(N):  
    if N <= 0:  
        return "O valor de N deve ser maior que 0"  
    elif N == 1:  
        return 1  
    elif N == 2:  
        return 1  
  
    sum, first, second = 1, 1, 1  
    count = 3  
  
    while count <= N:  
        sum = first + second  
        first = second  
        second = sum  
        count += 1  
  
    return sum
```

Convertendo Fibonacci em um algoritmo linear (2 de 2)

Usando o algoritmo, agora linear, para mostrar os n números de fibonacci

```
# Exemplo de uso:  
# n = a qtd de números da sequencia  
n = int(input("digite o valor de n: "))
```

```
for i in range(1, n+1):  
    print(fibonacci(i))
```

$O(n)$

Complexidade:

$O(n) * (n)$

$O(n^2)$

```
def fibonacci(N):  
    if N <= 0:  
        return "O valor de N deve ser maior que 0"  
    elif N == 1:  
        return 1  
    elif N == 2:  
        return 1
```

```
    sum, first, second = 1, 1, 1  
    count = 3
```

```
    while count <= N:  
        sum = first + second  
        first = second  
        second = sum  
        count += 1
```

```
    return sum
```

$O(n)$

VAMOS PARA A PRÁTICA ?!!!



Exercícios

1. **(ENADE)** Sobre recursão, assinale a alternativa correta:
 - A. Uma função recursiva **precisa obrigatoriamente** ter pelo menos dois casos base.
 - B. Se uma função recursiva não tiver caso base correto, ela **trava o programa** sem qualquer aviso.
 - C. Um algoritmo recursivo **sempre** é mais eficiente que um iterativo equivalente.
 - D. Toda função recursiva deve possuir um caso base e chamar a si mesma com um problema menor.

Exercícios

1. (ENADE) Sobre recursão, assinale a alternativa correta:

- A. Uma função recursiva **precisa obrigatoriamente** ter pelo menos dois casos base.
- B. Se uma função recursiva não tiver caso base correto, ela **trava o programa** sem qualquer aviso.
- C. Um algoritmo recursivo **sempre** é mais eficiente que um iterativo equivalente.
- D. Toda função recursiva deve possuir um caso base e chamar a si mesma com um problema menor.

Exercícios

2. (TEÓRICA) Considere a função Fibonacci recursiva definida acima. Qual o comportamento de tempo da versão recursiva conforme n cresce? Assinale a alternativa correta e justifique:

- A. $O(n)$ – pois cada número de Fibonacci é calculado em tempo constante.
- B. $O(n \log n)$ – devido à divisão do problema em duas chamadas menores.
- C. $O(2^n)$ – cada chamada gera duas subchamadas, levando a crescimento exponencial.
- D. $O(n^2)$ – pois cada chamada faz um loop aninhado até 2.

Exercícios

2. (TEÓRICA) Considere a função Fibonacci recursiva definida na aula. Qual o comportamento de tempo da versão recursiva conforme n cresce? Assinale a alternativa correta e justifique:

- A. $O(n)$ – pois cada número de Fibonacci é calculado em tempo constante.
- B. $O(n \log n)$ – devido à divisão do problema em duas chamadas menores.
- C. $O(2^n)$ – cada chamada gera duas subchamadas, levando a crescimento exponencial.
- D. $O(n^2)$ – pois cada chamada faz um loop aninhado até 2.

Exercícios

3. (PRÁTICA) Escreva uma função recursiva em Python chamada `soma_digitos(n)` que calcule a soma dos dígitos de um número inteiro positivo n . Por exemplo, `soma_digitos(123)` deve retornar $1 + 2 + 3 = 6$.

Exercícios

3. (PRÁTICA) Escreva uma função recursiva em Python chamada `soma_digitos(n)` que calcule a soma dos dígitos de um número inteiro positivo n . Por exemplo, `soma_digitos(123)` deve retornar $1 + 2 + 3 = 6$.

```
def soma_digitos(n):  
    if n < 10: # Caso base: número com um único dígito  
        return n  
    else:  
        ultimo_digito = n % 10  
        resto_do_numero = n // 10  
        return ultimo_digito + soma_digitos(resto_do_numero) # Passo recursivo  
  
print(soma_digitos(123))
```

Próxima Aula



- Estudar os **capítulos de 1 a 9** do livro “Estrutura de Dados com Python”



1ª AVALIAÇÃO / PROVA
Boa semana e bons estudos!!